Name- Vedant Wagh

Roll no.-C 47

Assignment 4

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct node
{
   int prn;
   char name[30];
   struct node *next;
};

/* Create a new node */
struct node* createNode()
{
   struct node *newNode;

   newNode = (struct node*)malloc(sizeof(struct node));

   printf("Enter PRN: ");
   scanf("%d", &newNode->prn);

   printf("Enter Name: ");
   scanf("%s", newNode->name);

   newNode->next = NULL;

   return newNode;
}

/* Add President */
struct node* addPresident(struct node *head)
{
   struct node *newNode = createNode();

   newNode->next = head;
   head = newNode;

   printf("President added successfully.\n");

   return head;
}

/* Add Secretary */
struct node* addSecretary(struct node *head)
{
   struct node *newNode = createNode();
   struct node *temp;
```

```
    if (head == NULL)
    {
        head = newNode;
    }
    else
    {
        temp = head;

        while (temp->next != NULL)
        {
            temp = temp->next;
        }

        temp->next = newNode;
    }

    printf("Secretary added successfully.\n");

    return head;
}

/* Add member at any position */
struct node* addMember(struct node *head)
{
    struct node *newNode, *temp;
    int pos, i;

    newNode = createNode();

    if (head == NULL)
    {
        head = newNode;
        return head;
    }

    printf("Enter position to add member: ");
    scanf("%d", &pos);

    if (pos == 1)
    {
        newNode->next = head;
        head = newNode;
    }
    else
    {
        temp = head;

        for (i = 1; i < pos - 1 && temp->next != NULL; i++)
        {
            temp = temp->next;
        }

        newNode->next = temp->next;
        temp->next = newNode;
    }
```

```
    printf("Member added successfully.\n");

    return head;
}

/* Delete President */
struct node* deletePresident(struct node *head)
{
    struct node *temp;

    if (head == NULL)
    {
        printf("Club is empty.\n");
        return head;
    }

    temp = head;
    head = head->next;

    printf("Deleted President: %s\n", temp->name);

    free(temp);

    return head;
}

/* Delete Secretary */
struct node* deleteSecretary(struct node *head)
{
    struct node *temp, *prev;

    if (head == NULL)
    {
        printf("Club is empty.\n");
        return head;
    }

    if (head->next == NULL)
    {
        printf("Deleted Secretary: %s\n", head->name);
        free(head);
        return NULL;
    }

    temp = head;

    while (temp->next != NULL)
    {
        prev = temp;
        temp = temp->next;
    }

    prev->next = NULL;
```

```
    printf("Deleted Secretary: %s\n", temp->name);

    free(temp);

    return head;
}

/* Delete member from a position */
struct node* deleteMember(struct node *head)
{
    struct node *temp, *prev;
    int pos, i;

    if (head == NULL)
    {
        printf("Club is empty.\n");
        return head;
    }

    printf("Enter position to delete: ");
    scanf("%d", &pos);

    if (pos == 1)
    {
        return deletePresident(head);
    }

    temp = head;

    for (i = 1; i < pos && temp != NULL; i++)
    {
        prev = temp;
        temp = temp->next;
    }

    if (temp == NULL)
    {
        printf("Invalid position.\n");
        return head;
    }

    prev->next = temp->next;

    printf("Deleted member: %s\n", temp->name);

    free(temp);

    return head;
}

/* Count members */
int countMembers(struct node *head)
{
    int count = 0;
    struct node *temp = head;
```

```
    while (temp != NULL)
    {
        count++;
        temp = temp->next;
    }

    return count;
}

/* Display members */
void display(struct node *head)
{
    struct node *temp = head;

    if (head == NULL)
    {
        printf("Club is empty.\n");
        return;
    }

    printf("\n--- Pinnacle Club Members ---\n");

    while (temp != NULL)
    {
        printf("PRN: %d\tName: %s\n", temp->prn, temp->name);
        temp = temp->next;
    }
}

/* Concatenate two lists */
struct node* concatenate(struct node *head1, struct node *head2)
{
    struct node *temp;

    if (head1 == NULL)
        return head2;

    temp = head1;

    while (temp->next != NULL)
    {
        temp = temp->next;
    }

    temp->next = head2;

    return head1;
}

/* Main function */
int main()
{
    struct node *head1 = NULL;
    struct node *head2 = NULL;
```

```
int choice, division, count;

do
{
   printf("\n===== PINNACLE CLUB =====\n");
   printf("1. Add President\n");
   printf("2. Add Member\n");
   printf("3. Add Secretary\n");
   printf("4. Delete President\n");
   printf("5. Delete Member\n");
   printf("6. Delete Secretary\n");
   printf("7. Count Members\n");
   printf("8. Display Members\n");
   printf("9. Concatenate Two Divisions\n");
   printf("10. Exit\n");

   printf("Enter your choice: ");
   scanf("%d", &choice);

   switch (choice)
   {
      case 1:
         printf("Enter Division (1 or 2): ");
         scanf("%d", &division);

         if (division == 1)
            head1 = addPresident(head1);
         else
            head2 = addPresident(head2);
         break;

      case 2:
         printf("Enter Division (1 or 2): ");
         scanf("%d", &division);

         if (division == 1)
            head1 = addMember(head1);
         else
            head2 = addMember(head2);
         break;

      case 3:
         printf("Enter Division (1 or 2): ");
         scanf("%d", &division);

         if (division == 1)
            head1 = addSecretary(head1);
         else
            head2 = addSecretary(head2);
         break;

      case 4:
         printf("Enter Division (1 or 2): ");
         scanf("%d", &division);
```

```
    if (division == 1)
        head1 = deletePresident(head1);
    else
        head2 = deletePresident(head2);
    break;

case 5:
    printf("Enter Division (1 or 2): ");
    scanf("%d", &division);

    if (division == 1)
        head1 = deleteMember(head1);
    else
        head2 = deleteMember(head2);
    break;

case 6:
    printf("Enter Division (1 or 2): ");
    scanf("%d", &division);

    if (division == 1)
        head1 = deleteSecretary(head1);
    else
        head2 = deleteSecretary(head2);
    break;

case 7:
    printf("Enter Division (1 or 2): ");
    scanf("%d", &division);

    if (division == 1)
        count = countMembers(head1);
    else
        count = countMembers(head2);

    printf("Total members = %d\n", count);
    break;

case 8:
    printf("Enter Division (1 or 2): ");
    scanf("%d", &division);

    if (division == 1)
        display(head1);
    else
        display(head2);
    break;

case 9:
    head1 = concatenate(head1, head2);
    head2 = NULL;

    printf("Two division lists concatenated successfully.\n");
    display(head1);
    break;
```

```
            case 10:
                printf("Exiting program...\n");
                break;

            default:
                printf("Invalid choice.\n");
        }

    } while (choice != 10);

    return 0;
}
```